NPTEL Software Testing (Advanced) Ultimate Exam Preparation Guide

Comprehensive Course Review with Step-by-Step Examples & 100 Exam-Style MCQs

Designed for Advanced Software Engineering Curriculums

Strictly aligned with NPTEL Syllabus & Structural Coverage Standards

Part 1: Comprehensive Theoretical Review with Concrete Examples

Topic 1: Input Space Partitioning (ISP)

Input Space Partitioning (ISP) is an elegant, black-box test design technique that models the input domain of the program under test. Testing every single possible input value is practically impossible. ISP resolves this challenge by partitioning the input domain into equivalence classes (called "blocks"), assuming that the program behaves similarly for any input value chosen from the same block.

Essential Rules of Partitioning

1. Completeness (Collectively Exhaustive): The union of all blocks b_i in partition P must cover the entire domain D. That is, $b_i = D$. No possible input can be left unclassified.
2. Disjointness (Mutually Exclusive): The intersection of any two different blocks b_i and b_j in partition P must be empty. That is, $b_i \cap b_j = \emptyset$ for all $i \neq j$. An input value cannot belong to more than one block in the same partition.

Modeling Approaches

- Interface-Based Approach: Focuses directly on the individual parameters as they appear in the program's signature (e.g., separating inputs strictly by type, size, or nullness in isolation). This approach is simple and easy to automate but can miss bugs arising from complex interactions between different parameters.
- Functionality-Based Approach: Focuses on the system's actual behavioral requirements and functional characteristics. It partitions the inputs based on their logical relationships, physical constraints, and semantic behavior under execution. This approach is highly effective at identifying subtle, multi-parameter bugs but requires deep domain knowledge.

Input Space Partitioning Coverage Criteria

- All-Combinations Coverage (ACoC): Every possible combination of blocks from all partitions is tested. If there are n partitions and each partition has $|P_i|$ blocks, the total number of test cases is the product: $\prod |P_i|$. This provides maximum thoroughness but suffers from combinatorial explosion.
- Each-Choice Coverage (ECC): Every individual block in every partition must be chosen in at least one test case. This is a very cheap criterion. If the partition sizes are $k_1, k_2, \ldots, k_n$, the minimum number of tests needed is simply $\max(|P_i|)$.
- Base-Choice Coverage (BCC): A highly popular exam topic. First, select a "base choice" (a standard, typical, or realistic value) for every single parameter/partition. Let this base test case be $T_{base} = [B_1, B_2, \ldots, B_n]$. To create the remaining test cases, vary exactly one parameter at a time to a non-base block, while holding all other parameters strictly at their base choices. Formula: $1 + \sum (d_i - 1)$.
- Multiple Base-Choice Coverage (MBCC): Similar to BCC, but recognizes that a single base choice may not cover all key usage patterns. It starts with multiple base choices and varies each parameter individually from each base choice.

Concrete ISP Example

Let's design test cases using Base-Choice Coverage (BCC) for a function: bookFlight(departureCity, destinationCity, cabinClass)

Step 1: Define Partitions & Blocks
1. Departure City (2 blocks): D_{base} (Valid domestic city), D_2 (Invalid city name)
2. Destination City (3 blocks): A_{base} (Valid domestic city), A_2 (Valid international city), A_3 (Invalid city name)
3. Cabin Class (3 blocks): C_{base} (Economy), C_2 (Business), C_3 (First Class)

Step 2: Construct the Base Case
• Test 1: [D_{base}, A_{base}, C_{base}] (Valid Domestic, Valid Domestic, Economy)

Step 3: Construct Alternative Cases (Varying one at a time)
• Vary Departure: Test 2: [D_2, A_{base}, C_{base}]
• Vary Destination: Test 3: [D_{base}, A_2, C_{base}], Test 4: [D_{base}, A_3, C_{base}]
• Vary Cabin: Test 5: [D_{base}, A_{base}, C_2], Test 6: [D_{base}, A_{base}, C_3]

Total BCC Tests = 1 + (2-1) + (3-1) + (3-1) = 6 tests.

Topic 2: Graph Fundamentals & Structural Coverage

Graph testing represents the software's execution flows or structures as a directed graph G = (N, E, N_0, N_f), where N is a set of Nodes, E is a set of Edges, N_0 is the set of Initial Nodes, and N_f is the set of Final Nodes.

- Path: A sequence of nodes [n_1, n_2, ..., n_k] of length k-1 such that there is a directed edge from n_i to n_{i+1}.
- Simple Path: A path where no node appears more than once, with the single exception that the first and last nodes of the path may be identical.
- Prime Path: A simple path that is not a subpath of any other simple path. This is a major structural target.
- Touring: A test path p tours a subpath q if q is a subpath of p.
- Touring with Sidetrips: A test path tours a subpath q with sidetrips if every node in q appears in p in the correct order, but there can be loop detours (sidetrips) in between.

Structural Coverage Criteria

- Node Coverage (NC): Visit every reachable node in the graph at least once. (Equivalent to Statement Coverage).
- Edge Coverage (EC): Visit every reachable edge in the graph at least once. (Equivalent to Branch Coverage).
- Edge-Pair Coverage (EPC): Visit every reachable path of length 2 in the graph.

- Prime Path Coverage (PPC): Tour every single prime path in the graph. PPC guarantees Node, Edge, and Edge-Pair Coverage.

Step-by-Step Prime Path Calculation

Let's find all the prime paths for the graph: N = {1, 2, 3, 4}, E = {(1,2), (2,3), (3,2), (2,4)}

Step 1: Simple paths of length 0 (Nodes): [1], [2], [3], [4]
Step 2: Simple paths of length 1 (Edges): [1, 2], [2, 3], [3, 2], [2, 4]
Step 3: Simple paths of length 2: [1, 2, 3], [1, 2, 4], [2, 3, 2], [3, 2, 3], [3, 2, 4]
Step 4: Simple paths of length 3: [1, 2, 3, 2] (NOT simple, node 2 repeated internally)

Step 5: Identify Prime Paths (not subpaths of any other simple path):
1. [1, 2, 3]
2. [1, 2, 4]
3. [2, 3, 2]
4. [3, 2, 3]
5. [3, 2, 4]

Topic 3: Control Flow Graphs (CFG) & Source Code Coverage

A Control Flow Graph (CFG) represents the execution flow of program statements. A Basic Block is a sequence of consecutive statements with a single entry point and a single exit point. No branching occurs inside a basic block; if the first statement executes, all subsequent statements execute in order.

- If-Then-Else: A decision node with two exiting edges representing the true and false branches. They merge at a join node.
- While/For Loop: The loop header is a decision node. It has a true edge leading into the loop body, and a false edge leading out of the loop.
- Statement vs. Branch Coverage: Statement Coverage (C_0) measures the percentage of executable statements executed. Branch Coverage (C_1) measures the percentage of decision outcomes exercised. Branch Coverage strictly subsumes Statement Coverage.

Topic 4: Data-Flow Testing

Data-Flow testing focuses on the lifecycle of variables. It tracks how values are defined (written) and where they are used (read) to detect anomalies.

- Definition (def): A location where a value is stored into a variable (e.g., x = 5;).
- Use: A location where the value of a variable is read. c-use (computation use) occurs in a computation statement (associated with nodes). p-use (predicate use) directly affects the flow of control in a decision (associated with edges).
- Def-Clear Path: A path from node n_i to node n_j for variable x where x is not redefined at any intermediate node or edge.

- du-pair: A pair (d, u) where d is a def of x, and u is a use of x, connected by at least one def-clear path.

Topic 5: Logic Coverage (PC, CC, CoC)

Logic coverage applies to boolean expressions (predicates) containing individual boolean variables or relational expressions (clauses).

- Predicate Coverage (PC): The entire predicate P evaluates to True at least once and False at least once. (Equivalent to Branch Coverage).
- Clause Coverage (CC): Each individual clause c in the predicate evaluates to True at least once and False at least once. Note: CC does NOT guarantee PC!
- Combinatorial Coverage (CoC): Tests all possible 2^n combinations of truth values for the n clauses.

Topics 6 & 7: Active & Inactive Clause Coverage

The Principle of Determination

A major clause c_i determines a predicate P if and only if changing the truth value of c_i causes the overall truth value of P to flip, while keeping all other clauses constant.
Mathematically: P_{c_i} = P[c_i True] P[c_i False] = True

- General Active Clause Coverage (GACC): c_i must determine P. Minor clauses do not need to have the same truth values when c_i is True vs False.
- Correlated Active Clause Coverage (CACC): c_i must determine P, and the overall outcome of the predicate P must change.
- Restricted Active Clause Coverage (RACC): c_i must determine P, and the minor clauses must have the exact same truth values in both test cases.

Topics 8 & 9: Mutation Testing & Object-Oriented Mutation

Mutation Score Formula

Mutation Score = [Killed Mutants / (Total Mutants - Equivalent Mutants)] × 100%

Detecting equivalent mutants is mathematically undecidable (reduces to the Halting Problem).

- Mutant: A copy of the original program containing a single, tiny syntactic change.
- Killed Mutant: A mutant is killed if at least one test case produces a different output compared to the original program.
- Equivalent Mutant: A mutant that behaves exactly identical semantically to the original program under all possible inputs.
- OO Mutation Operators: Polymorphic shift, access modifier change, super-class call removal, and parameter type casting.

Topic 10: Specification, FSM & OO Testing

- Finite State Machine (FSM) Testing: Systems are modeled as states, transitions, triggers, and actions. Coverage criteria include State Coverage, Transition Coverage, and All-Round-Trips Coverage.
- Object-Oriented (OO) Testing: Focuses on unique OO traits like Encapsulation (verifying internal states cannot be corrupted) and Inheritance & Polymorphism.

Topic 11: Regression Testing

- Test Suite Minimization: Permanently removes redundant test cases that cover identical code structures.
- Test Case Selection: Selects a subset of existing test cases affected by specific code modifications.
- Test Case Prioritization: Orders test cases to maximize the rate of fault detection (measured using APFD).

Topic 12: Non-Functional Testing

- Performance & Load Testing: Evaluates responsiveness and stability under typical and peak workloads.
- Stress Testing: Pushes the system beyond its absolute limit until it breaks, observing recovery behavior.
- Security & Usability Testing: Evaluates vulnerability to attacks and human user experience.

Topic 13: Web & Mobile Application Testing

- Web Testing: Addresses browser compatibility, responsive layout design, and network latencies.
- Mobile Testing: Addresses severe environmental factors like Interruption Testing (calls, SMS), Network Swapping (Wi-Fi to cellular), and Hardware Interaction.

Part 2: 100 NPTEL Exam-Style Multiple Choice Questions

Input Space Partitioning (ISP)

Q1. Which of the following is a primary requirement for a valid partition in Input Space Partitioning?

A. Complete coverage and disjointness
B. Overlapping blocks to ensure boundary checks
C. High mutation score
D. A minimum of 5 distinct characteristics

Q2. If a partition of a domain is NOT disjoint, which of the following is violated?

A. Completeness
B. Disjointness
C. Structural coverage
D. Def-clear flow

Q3. In interface-based input space partitioning, characteristics are selected based on:

A. The functional requirements of the program
B. The internal code logic and variables
C. The parameters in isolation as they appear in the interface
D. The state machine transitions

Q4. Functionality-based input space partitioning is typically more powerful than interface-based because:

A. It requires no knowledge of the requirements
B. It can identify relationships and semantic constraints between different parameters
C. It always yields fewer test cases
D. It guarantees statement coverage

Q5. Consider a function with 3 parameters. Partition 1 has 3 blocks, Partition 2 has 4 blocks, and Partition 3 has 2 blocks. How many test cases are required for All-Combinations Coverage (ACoC)?

A. 9
B. 12
C. 24
D. 18

Q6. For the same partitions in Q5, what is the minimum number of test cases required for Each-Choice Coverage (ECC)?

A. 2

B. 3

C. 4

D. 24

Q7. For the same partitions in Q5, what is the exact number of test cases required for Base-Choice Coverage (BCC)?

A. 7

B. 9

C. 24

D. 10

Q8. Which coverage criterion is strictly subsumed by All-Combinations Coverage (ACoC)?

A. Only Each-Choice Coverage

B. Only Base-Choice Coverage

C. Both Each-Choice and Base-Choice Coverage

D. Neither

Q9. In Base-Choice Coverage, how do we construct non-base test cases?

A. By varying all parameters simultaneously

B. By varying exactly one parameter at a time from the base choice

C. By keeping all parameters at their worst-case values

D. By randomly selecting values from each partition

Q10. What is a major disadvantage of Multiple Base-Choice Coverage (MBCC) compared to standard BCC?

A. It fails to test the base choices

B. It requires fewer test cases

C. It increases the total number of test cases, requiring more effort

D. It cannot be used with functionality-based ISP

Graph Fundamentals & Structural Coverage

Q11. A directed graph is defined as G = (N, E, N_0, N_f). What does N_f represent?

A. Node frequency
B. Neighbor functions
C. Set of final nodes
D. Flow factors

Q12. A path [n_1, n_2, ..., n_k] is simple if:

A. It has a length of 1
B. No node is repeated, except possibly the first and last nodes
C. Every node is visited twice
D. It contains no edges

Q13. What is a prime path?

A. A path containing a prime number of nodes
B. A simple path that is not a subpath of any other simple path
C. The shortest path between the start node and the final node
D. A path that cannot be toured

Q14. Node Coverage in graph testing is conceptually equivalent to which code coverage metric?

A. Branch Coverage
B. Statement Coverage
C. Path Coverage
D. Mutation Coverage

Q15. Edge Coverage strictly subsumes:

A. Node Coverage
B. Edge-Pair Coverage
C. Prime Path Coverage
D. Complete Path Coverage

Q16. If a test path p visits every node and edge in a graph, does it guarantee Edge-Pair Coverage?

A. Yes, always
B. No, because Edge-Pair Coverage requires visiting consecutive pairs of edges
C. Yes, if the graph has no loops
D. Yes, only if the graph is acyclic

Q17. Under what condition is Prime Path Coverage (PPC) impossible to achieve with a finite number of test cases?

A. If the graph contains a loop

B. PPC can always be achieved with a finite number of test cases, even with loops, because the set of prime paths is always finite

C. If the graph has multiple final nodes

D. If the graph has no entry node

Q18. What is "touring with a sidetrip"?

A. Bypassing a node completely

B. Permitting a loop detour from a node in the target path before returning to continue

C. Testing a completely unrelated graph

D. Skipping the final node

Q19. If a graph has nodes {1, 2, 3} and edges {(1,2), (2,3)}, how many prime paths exist?

A. 1

B. 2

C. 3

D. 4

Q20. For the graph in Q19, what is the longest prime path?

A. [1, 2]

B. [2, 3]

C. [1, 2, 3]

D. [1, 3]

Control Flow Graphs (CFG) from Source Code

Q21. What constitutes a "Basic Block" in a Control Flow Graph?

A. Any statement containing an if condition

B. A sequence of statements with a single entry point and a single exit point, containing no internal branching

C. A block of code containing at least one loop

D. The entire body of a class declaration

Q22. In a CFG, an if-then-else structure typically results in:

A. A single node with a self-loop
B. A decision node with two exiting edges merging later at a join node
C. Three disjoint graphs
D. An infinite loop representation

Q23. Which of the following code structures introduces a loop back-edge in a CFG?

A. switch-case
B. if-else
C. while loop
D. Sequential variable assignments

Q24. If a test suite achieves 100% Branch Coverage, what is the guaranteed Statement Coverage?

A. 50%
B. 0%
C. 100%
D. It depends entirely on the compiler

Q25. Can 100% Statement Coverage be achieved without executing all branches?

A. Yes, because empty else branches do not contain statements to be covered
B. No, they are mathematically identical
C. Yes, but only in Java programs
D. No, statement coverage is stronger than branch coverage

Q26. For a simple program with no branching or loops, what is the minimum number of test cases needed to achieve both Statement and Branch Coverage?

A. 0
B. 1
C. 2
D. 3

Q27. How is a switch-case statement represented in a CFG?

A. As a single linear path
B. As a node with multiple outgoing edges corresponding to each case
C. As an nested set of self-loops
D. It cannot be represented in a standard CFG

Q28. What does a "join node" represent in a CFG?

A. The entry point of the main function

B. The point where different execution paths (e.g., from an if-else) merge back together

C. The termination point of the program

D. The declaration of a global variable

Q29. In a CFG, how is a for loop condition node connected?

A. It has only one incoming and one outgoing edge

B. It has an edge to the loop body, an edge exiting the loop, and an incoming edge from the end of the loop body

C. It is isolated from the rest of the program

D. It has no edges

Q30. CFG-based testing is classified under which of the following?

A. Black-box testing

B. White-box testing

C. Performance testing

D. Usability testing

Data-Flow Testing

Q31. In Data-Flow testing, a variable is "defined" (def) when it is:

A. Read from memory

B. Assigned a value or initialized

C. Passed as a constant argument

D. Declared without any value

Q32. What is a "use" of a variable in Data-Flow testing?

A. When the variable is garbage collected

B. When the variable is read or referenced in a computation or decision

C. When the variable is redefined

D. When the variable goes out of scope

Q33. Which of the following is an example of a c-use (computation use)?

A. if (x > 0)
B. y = x * 10;
C. while (x == y)
D. return (x != null);

Q34. Which of the following is an example of a p-use (predicate use)?

A. z = a + b;
B. if (status == true)
C. print(x);
D. y = Math.sqrt(z);

Q35. A path [n_1, n_2, ..., n_k] is "def-clear" for a variable x if:

A. Variable x is defined at every node in the path
B. Variable x is not reassigned a new value at any intermediate node or edge
C. Variable x is set to null
D. The path contains no loops

Q36. What is a "du-pair"?

A. A double use of a variable in the same statement
B. A pair of definition and use locations for a variable connected by at least one def-clear path
C. Two definitions of the same variable in a block
D. A pair of duplicate variable declarations

Q37. Which data-flow coverage criterion is the strongest?

A. All-Defs Coverage
B. All-Uses Coverage
C. All-du-paths Coverage
D. All-c-uses Coverage

Q38. All-Defs Coverage requires that we test:

A. Every single use of a variable
B. At least one def-clear path from every definition to some use of that variable
C. Every possible path in the program
D. Only the global variables

Q39. If a variable x is defined at node 1, and used at nodes 3 and 4, how many du-pairs exist for x assuming def-clear paths exist?

A. 1
B. 2
C. 3
D. 4

Q40. Data-flow anomalies (like using an uninitialized variable) are best detected using:

A. Logic coverage
B. Data-flow testing and analysis
C. Equivalence partitioning
D. Stress testing

Logic Coverage

Q41. For a predicate P = A B, what is the minimum number of test cases required for Predicate Coverage (PC)?

A. 1
B. 2
C. 4
D. 8

Q42. For the predicate P = A B, which of the following test suites satisfies Clause Coverage (CC)?

A. {TT, FF}
B. {TF, FT}
C. Both A and B satisfy Clause Coverage
D. Neither satisfies Clause Coverage

Q43. Why does Clause Coverage (CC) NOT guarantee Predicate Coverage (PC)?

A. Because CC requires more test cases than PC
B. Because we can satisfy CC while keeping the overall predicate evaluation constant (never toggling)
C. Because CC is a black-box technique
D. Because CC applies only to loops

Q44. How many test cases are required for Combinatorial Coverage (CoC) of a predicate with 4 independent clauses?

A. 4

B. 8

C. 16

D. 32

Q45. Combinatorial Coverage is often impractical in real-world testing because:

A. It does not find any bugs

B. The number of test cases grows exponentially (2^n) with the number of clauses

C. It cannot be automated

D. It is only applicable to simple statements

Q46. A predicate is defined as P = A B C. If we use the truth values A=True, B=False, C=False, what is the value of P?

A. False

B. True

C. Undetermined

D. Null

Q47. If a test suite achieves 100% Combinatorial Coverage, does it guarantee Predicate Coverage?

A. No, never

B. Yes, always

C. Only if the predicate has less than 2 clauses

D. Only in the absence of boolean operators

Q48. Which of the following is a clause in the predicate P = (x > 5) (y < 2)?

A. (x > 5)

B.

C. P

D. (x > 5) (y < 2)

Q49. Which logic coverage criterion is considered the weakest?

A. Predicate Coverage

B. Clause Coverage

C. Combinatorial Coverage

D. All of them are of equal strength

Q50. If we have a predicate with 3 clauses, what is the maximum possible number of distinct rows in its truth table?

A. 3

B. 6

C. 8

D. 9

Active & Inactive Clause Coverage (ACC / ICC)

Q51. A clause c_i is said to "determine" a predicate P if:

A. P is always True when c_i is True

B. Changing the truth value of c_i flips the truth value of the overall predicate P, keeping other clauses constant

C. c_i is the only clause in the predicate

D. c_i has a value of False

Q52. The mathematical formula to check if clause c determines predicate P is:

A. $P_{c=True}$ $P_{c=False}$

B. $P_{c=True}$ $P_{c=False}$ = True

C. $P_{c=True}$ $P_{c=False}$ = False

D. $P_{c=True} = P_{c=False}$

Q53. In Correlated Active Clause Coverage (CACC), the minor clauses must be chosen such that:

A. The major clause determines the predicate, and the predicate outcome toggles between True and False

B. The minor clauses remain constant in all possible tests

C. The major clause evaluates to True in all test cases

D. The predicate remains False throughout

Q54. What makes Restricted Active Clause Coverage (RACC) "restricted" compared to GACC?

A. It restricts the number of active clauses to one

B. It requires that the minor clauses have the exact same truth values in both the True and False test cases for the major clause

C. It can only be applied to predicates with two clauses

D. It cannot be used with OR gates

Q55. Which of the following is a major advantage of RACC over GACC?

A. It completely isolates the major clause, ensuring any change in output is directly attributable to that clause

B. It requires fewer test cases

C. It does not require calculating determination

D. It is a purely manual technique

Q56. If a clause does NOT determine a predicate, we can still test it using:

A. Active Clause Coverage

B. Inactive Clause Coverage

C. Combinatorial Coverage

D. Edge-Pair Coverage

Q57. In General Inactive Clause Coverage (GICC), our goal is to show that:

A. The major clause determines the predicate

B. The major clause has no effect on the predicate outcome, and the overall predicate evaluates to the same value regardless of the major clause's value

C. All clauses are True

D. The program crashes when the major clause is modified

Q58. For the predicate P = A B, does clause A determine P if B = False?

A. Yes, because if A changes, P changes

B. No, because if B = False, P is always False regardless of the value of A

C. Yes, under GACC rules only

D. Yes, if A = True

Q59. For the predicate P = A B, under what condition does clause A determine P?

A. When B = True

B. When B = False

C. Under any condition

D. Never

Q60. If we satisfy RACC for all clauses in a predicate, we have also guaranteed:

A. CACC and GACC

B. Only GACC

C. Only Combinatorial Coverage

D. None of the above

Mutation Testing

Q61. What is the primary purpose of Mutation Testing?

A. To find bugs in the original production code
B. To evaluate the quality and thoroughness of a test suite by introducing artificial faults
C. To speed up code execution
D. To automate the generation of test cases

Q62. A "mutant" program is:

A. A completely rewritten version of the software in a different language
B. A copy of the original program containing a single, small syntactic change
C. A program that contains compilation errors
D. A high-performance version of the program

Q63. A mutant is said to be "killed" if:

A. It fails to compile
B. At least one test case produces a different output on the mutant program than on the original
C. It runs slower than the original program
D. It is manually deleted by the tester

Q64. If a mutant has the exact same semantic behavior as the original program under all circumstances, it is called a/an:

A. Live mutant
B. Equivalent mutant
C. Dead mutant
D. Active mutant

Q65. Why is finding equivalent mutants considered "undecidable"?

A. Because they do not exist in modern languages
B. Because it mathematically reduces to the Halting Problem
C. Because it requires too much CPU memory
D. Because testers are not trained in logic

Q66. What is the mutation score of a test suite if it kills 80 out of 100 mutants, where 10 mutants are proven to be equivalent?

A. 80%

B. 88.89%

C. 90%

D. 72.5%

Q67. Which of the following is a standard mutation operator for source code?

A. Swapping '+' with '-'

B. Changing 'x > y' to 'x >= y'

C. Swapping boolean constants true and false

D. All of the above

Q68. A mutant remains "live" if:

A. The test suite fails to detect the difference between the mutant and the original program

B. The mutant crashes during execution

C. The test suite kills it instantly

D. The mutant is equivalent

Q69. Strong Mutation testing requires:

A. Simply executing the mutated statement

B. The mutant to affect the program state and propagate a different output to the program termination

C. Modifying the program's binary code directly

D. Having a mutation score of 100%

Q70. Weak Mutation testing differs from Strong Mutation because it only requires:

A. The mutated component to show a different state immediately after execution of the mutated statement

B. Testing of global variables

C. No compilation of the mutant

D. A mutation score of 0%

Integration & OO Mutation

Q71. Integration and OO Mutation testing focuses primarily on mutating:

A. Simple arithmetic operators inside a method
B. The relationships, interfaces, inheritance, and polymorphic interactions between classes
C. The operating system kernel
D. Database query structures

Q72. Which of the following is a typical OO mutation operator?

A. Deleting or modifying a super class method call
B. Changing class access modifiers (e.g., private to public)
C. Swapping polymorphic object instantiations
D. All of the above

Q73. In OO mutation, what does a "Polymorphic Shift" operator do?

A. It changes the parent class of an object to an unrelated interface
B. It changes the dynamic type of an object to another compatible class in the inheritance hierarchy
C. It deletes the class constructor
D. It converts a class into a struct

Q74. Mutating access modifiers (e.g., from private to public) helps verify if:

A. The compiler is functioning correctly
B. The test suite can catch encapsulation violations and visibility issues
C. The execution speed improves
D. The database is secure

Q75. OO mutation testing is highly important because:

A. Traditional mutation operators do not target structural OO bugs like inheritance mismatches or polymorphic binding faults
B. OO programs do not use arithmetic operators
C. It is much faster than unit mutation
D. It requires no execution of test cases

Specification-Based, FSM, and OO Testing

Q76. Finite State Machine (FSM) testing is classified as:

A. White-box testing only

B. Specification-based (black-box or gray-box) testing

C. Performance testing

D. Static code analysis

Q77. What does "State Coverage" in FSM testing require?

A. That all lines of code are executed

B. That the test cases visit every state in the state machine at least once

C. That the program runs in all states of India

D. That all variables are defined

Q78. "Transition Coverage" in FSM testing requires that:

A. Every state transition in the machine is exercised at least once

B. No transitions are allowed

C. We test the speed of transitions

D. Only the initial and final states are visited

Q79. Object-Oriented testing must address "Encapsulation" by verifying that:

A. Class variables can be accessed from anywhere

B. The internal state of an object can only be modified through its authorized public methods

C. No classes inherit from each other

D. Classes are written in single files

Q80. In OO testing, inheritance can introduce issues because:

A. Subclasses do not inherit any behavior

B. Inherited methods might behave unexpectedly when executed in the context of the subclass's new state variables

C. Constructors cannot be inherited

D. It forces all methods to be static

Regression Testing

Q81. The main objective of Regression Testing is to:

A. Test the program's performance under heavy load

B. Ensure that modifications, updates, or bug fixes have not broken existing, working functionality

C. Delete older versions of the source code

D. Write a completely new functional specification

Q82. Test Suite Minimization (Reduction) is the process of:

A. Deleting all test cases to save space

B. Identifying and permanently removing redundant test cases that cover identical program structures

C. Converting test cases into comments

D. Only running tests that fail

Q83. Test Case Selection differs from Minimization because:

A. Selection is temporary for a specific change, while Minimization is a permanent reduction of the suite

B. Selection is only done by managers

C. Minimization requires no knowledge of the code

D. Selection is always random

Q84. What is the goal of Test Case Prioritization?

A. To minimize the size of the test suite

B. To order the execution of test cases so that those with higher benefit (like faster fault detection) run first

C. To run all test cases simultaneously

D. To prioritize manual tests over automated ones

Q85. APFD (Average Percentage of Faults Detected) is a metric used to evaluate:

A. The execution speed of the compiler

B. The effectiveness of a prioritized test suite at exposing faults early in its execution

C. The number of lines of code in the software

D. The security level of a network

Q86. An APFD score of 95% indicates that:

A. 95% of the code is covered

B. The prioritized test suite detects the vast majority of faults very early in its execution run

C. 5% of the tests failed

D. The program is bug-free

Non-Functional Testing

Q87. Non-functional testing evaluates:

A. What the system does

B. How well the system performs under specific operational and environmental constraints

C. Only the syntax of the program

D. The database schema design

Q88. Which of the following is a type of Non-Functional testing?

A. Unit testing

B. Usability testing

C. Integration testing

D. Logic coverage

Q89. Load Testing is designed to evaluate system behavior under:

A. Zero active users

B. Anticipated, typical, or peak functional workloads

C. Volatile, catastrophic system failures

D. Invalid input environments

Q90. Stress Testing differs from Load Testing because Stress Testing:

A. Only tests the database

B. Pushes the system beyond its specified operational limit until it breaks to observe recovery behavior

C. Is performed in shorter durations

D. Focuses strictly on code syntax

Q91. Security testing is primarily concerned with identifying:

A. Memory leaks

B. Vulnerabilities, threats, and risks of malicious attacks or unauthorized access

C. Slow database queries

D. Complex code paths

Q92. Usability testing evaluates:

A. The number of users accessing the system

B. How easily and intuitively a human user can interact with, navigate, and operate the application

C. The compiler efficiency

D. The execution path coverage

Web & Mobile Application Testing

Q93. Web application testing must heavily account for:

A. Device battery drains only

B. Browser rendering engine differences, responsive layouts, and varying network latencies

C. Standard unit testing only

D. Hard drive sector storage errors

Q94. Responsive layout testing verifies that:

A. The server responds within 2 seconds

B. The user interface adjusts dynamically and looks correct across various screen sizes and resolutions

C. The database can handle multiple tables

D. The code is structured into modules

Q95. What is a key challenge unique to Mobile Application Testing compared to desktop software?

A. Mobile apps do not have databases

B. Severe environmental variability such as network drops, physical device interruptions, and hardware limitations

C. They cannot be tested using black-box techniques

D. They do not use operating systems

Q96. Interruption Testing in mobile apps verifies how the app handles:

A. Running out of internet data

B. Sudden incoming calls, SMS messages, low battery warnings, or push notifications during active app use

C. Changing variable names in the code

D. Pressing back buttons repeatedly

Q97. Testing a mobile app's behavior when transitioning from Wi-Fi to a 4G/5G cellular network is an example of:

A. Boundary value analysis

B. Network swapping (connectivity state transition) testing

C. Static analysis

D. Mutation testing

Q98. Resource consumption testing in mobile devices evaluates:

A. How the app affects battery life, memory usage, and local storage limits

B. The number of developers needed to write the app

C. The cost of hosting the app in an app store

D. The speed of the server CPU

Q99. Which testing approach involves running mobile apps on simulated software profiles on a PC?

A. Physical device testing

B. Emulator/Simulator testing

C. Field testing

D. Mutation testing

Q100. Compatibility testing for mobile applications checks if:

A. The app works on various OS versions (e.g., Android 11 vs. Android 13) and screen sizes

B. The code compiles without warnings

C. The development environment is compatible with the server

D. The database queries run in constant time

Part 3: Official Answer Key

Question	Answer	Question	Answer	Question	Answer	Question	Answer
Q1	A	Q26	B	Q51	B	Q76	B
Q2	B	Q27	B	Q52	B	Q77	B
Q3	C	Q28	B	Q53	A	Q78	A
Q4	B	Q29	B	Q54	B	Q79	B
Q5	C	Q30	B	Q55	A	Q80	B
Q6	C	Q31	B	Q56	B	Q81	B
Q7	C	Q32	B	Q57	B	Q82	B
Q8	C	Q33	B	Q58	B	Q83	A
Q9	B	Q34	B	Q59	B	Q84	B
Q10	C	Q35	B	Q60	A	Q85	B
Q11	C	Q36	B	Q61	B	Q86	B
Q12	B	Q37	C	Q62	B	Q87	B
Q13	B	Q38	B	Q63	B	Q88	B
Q14	B	Q39	B	Q64	B	Q89	B
Q15	A	Q40	B	Q65	B	Q90	B
Q16	B	Q41	B	Q66	B	Q91	B
Q17	B	Q42	C	Q67	D	Q92	B
Q18	B	Q43	B	Q68	A	Q93	B
Q19	C	Q44	C	Q69	B	Q94	B
Q20	C	Q45	B	Q70	A	Q95	B
Q21	B	Q46	B	Q71	B	Q96	B
Q22	B	Q47	B	Q72	D	Q97	B
Q23	C	Q48	A	Q73	B	Q98	A
Q24	C	Q49	B	Q74	B	Q99	B
Q25	A	Q50	C	Q75	A	Q100	A